FEUP - Master in Informatics and Computing Engineering

Interactive Graphics Systems

2025-2026

Practical classes

Practical Work 1

An introductory application in Three.js

Introduction

Interactive 3D graphics can be generated directly within web browsers through the use of several established technologies. Among these, **WebGL** is particularly relevant, as it is built upon OpenGL (specifically OpenGL ES 2.0/3.0) and provides a low-level programming interface. A higher-level alternative is, among others, **Three.js** (https://threejs.org/) which is based on JavaScript and WebGL.

In this course, you will study and analyse Interactive Graphic Systems concepts, developing graphic applications using Three.js and JavaScript that can be run in recent web browsers. Development will take place in the practical classes and, for file and version control we will use the UP Gitlab system (gitlab.up.pt). The Moodle website contains detailed information about the initial configuration of this system, which should be taken into account before proceeding.

Objectives of Practical Work TP1

This work focuses on the design of primitives, their

and start using the Three.js technology, and to study and to expand the base examples provided. The main topics to be covered are:

- Study of the architecture of an application in Three.js
- Create simple objects (meshes, from primitives);
- Create groups (aggregations of other groups or meshes)
- Affect geometric transformations to existing objects/groups;
- Affect visualization properties to existing objects;
- Create and use a graphical interface (GUI) to control aspects of the scene and its objects.

The ultimate goal is for you to get familiar with these concepts, that will be the basis for the experiments and application of concepts, to be performed later on in this course

Pre-Requisites

It is assumed that you have prior knowledge of the basic Computer Graphics concepts from previous courses, such as 3D transformations, lighting, materials and textures.

If that is not the case, you should explore those topics in advance. You can find documentation and exercises regarding those topics in the Moodle section titled “Computer Graphics Basics”.

Your teacher can help out in pointing other resources and clarifying doubts, but there will not be specific class time devoted to those topics.

Moodle quiz and screenshots

Throughout this work there are short observation questions that you will have to answer on a questionnaire in moodle.

Each question will require you to fill the answer in a textbox, as well as upload one/two image files of screenshots supporting/illustrating your answer.

Questions are identified throughout this document with the icon and the screenshots with the icon. The evaluation details about the questionnaire are described further, in section Evaluation.

Final submission

Exercise base code

The base code is provided in the GIT project corresponding to your workgroup. As you can see in Figure 1, the resulting 3D scene is composed of:

- Three colored thin cones representing an axis system ***xyz***;
- A cube centered and located on the positive part of the axis of ***yy***;
- A plane centered on the origin of the axes;
- One point light source illuminating the scene from above (out of view).

The cube and the plane have different materials.

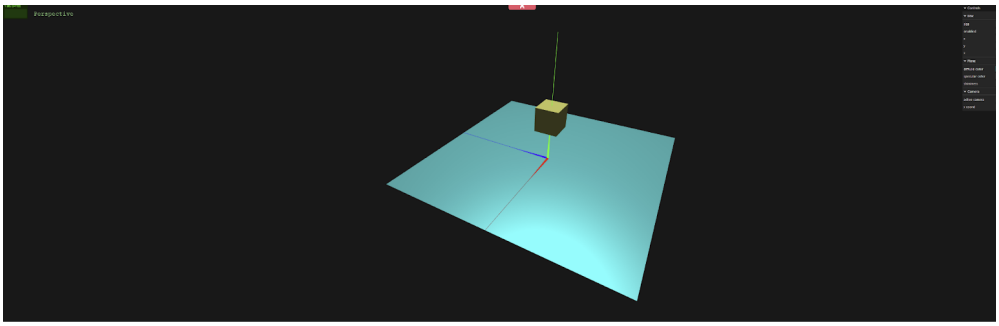


Figure 1: Illustration of the visual result corresponding to the provided base code.

In the upper right corner, there is an area where a 2D interface is implemented. Through this interface, you can activate or parameterize some aspects of the scene. Mouse interaction is also available, functioning as follows:

- Left button - rotate the scene around the origin point of the coordinates;
- Scroll wheel - zoom in/out (alternatively: CTRL + Left button);
- Right button - “slide” the camera sideways (pan).

The base source code provided for the exercise is grouped into several files that describe interconnected classes and instances (objects). The files are, in particular:

- **index.html**: application startup point; This file contains the HTML, CSS structure and imports the JS libraries necessary to run the application.
- **main.js**: Javascript starting point, invoked as a module in index.html; builds and initializes

representation of the axis system, supported on a *helper* [1]and in 3 instances of the cone primitive. An instance of the MyAxis class is created by building the scene in MyContents.js.

- **MyGuiInterface.js**: 2D GUI instance (MyGuiInterface.js)

NOTE: some of these objects are interconnected, since the content object and the 2D graphical interface need to know the application object and vice versa.

The source code provides instructions for the construction of a **WebGLRenderer** and its association with an HTML element with an area (for example, a) where the visual result generated by Three.js is presented.

After building the application, the main display cycle is run repeatedly by calling the method *render()*, provided for in **MyApp**.

The class **MyGuiInterface** implements the 2D graphical interface where text inputs, checkboxes, sliders, drop-down menus , among other elements, can be used to interact with the created scene. The interface uses the **dat.GUI** library, for which more information is available at: https://github.com/dataarts/dat.gui/blob/master/API.md

Carefully read the comments available in the source code as they provide important information about its operation and use.

Notes to the Git repository for projects

The source code developed throughout the practical classes, must be maintained in an online repository, hosted in UP's Gitlab. Students are organized into groups of three. In the first class, each group will be assigned their own Gitlab repository that already contains the base source code for this work, described previously.
More information about using UP's Gitlab, as well as preparing the work environment, is available on the Moodle page.

Work Components

The components of the work are described in the following sections, to be developed over the next

The code provides the use, by default, of a "Perspective" type camera, although it is possible, through the graphical interface, to change it to "Orthographic" with three possible points of view. Identify the files related to the camera and:

- Manipulate the parameters of the `PerspectiveCamera's` constructor
- Manipulate the position of the camera
- Create another perspective camera, with a different point of view
- Analyse the code section that creates the orthogonal cameras, and take notice of their parameters: *left*, *right*, *top*, ... *far*, and also `*.up, *position, *.lookAt()`
- Create two new orthogonal cameras for the `right` and `back` views
- Add the new cameras created in the previous points in the interface in the camera selection menu.

2. Geometric Transformations

Three.js technology uses matrices to represent 3D geometric transformations: translation (position), rotation and scaling. All 3D objects in Three.js (which inherit from the Object3D class) contain an attribute *matrix* representing the position (`position`), the rotation (`rotation`) and the scaling (`scale`) of the object.

Find the functions that create and manipulate the *box* object in the scene:

- Change the `scale` property directly to `(3,2,1)`, before the lines manipulating the rotation and position. Change the rotation angle to 30º.
- Invert the order of these changes, so that position is changed first, then rotation and scale. Compare the visual results with the previous task.
  (2.1)

Moodle Task - PW1-A

On Moodle, discuss the following question and submit this section's screenshot (2.1):

Did you observe any differences when changing the order of the lines manipulating the transformations?

`rotateX()` function calls (same angle). Compare the visual results to the previous task.
(2.2)

Note: The position of the box is being manipulated each frame, so you might not see changes if you try to manipulate it when it is created.

Moodle Task - PW1-B

On Moodle, discuss the following question and submit this section's screenshots (2.2):

What were the differences between setting the rotation property and calling the rotateX() function? Did it behave as it would in WebGL/WebCGF?

3. Geometric Primitives (Geometries)

Three.js technology provides a wide variety of predefined geometries, most of which are 3D geometries, which we call geometric primitives.

In this part of the work, you will be asked to explore different geometric primitives by including them in your code and testing them in their various aspects (e.g. dimensions, angles, number of stacks, number of slices, etc.). Resorting to the documentation, explore the following (you can try others to your choosing):

- Plane
- Circle
- Box
- Sphere (complete and partial)
- Cylinder (complete and partial)
- Cone (complete and partial)
- Polyhedron (no subdivisions)

Modeling a 3D scene

Model a scene containing the following set of objects:

- Floor (you can take advantage of the existing plane)
- Four walls (planes)
- A table centered on the scene, composed by:

- create an "empty" object *tableObj* of type **THREE.Object3D**,
- add the table components (tabletop, legs) to that object, instead of adding to the scene
- Add the *tableObj* to the scene

To better organize your code, you can make your table an actual subclass of Object3D. Follow the example of the MyAxis object/class, and create a MyTable class.

In addition to the above mentioned objects, add some objects of your choosing for decoration:

- At least 1 compound object placed over the table, composed of 2+ primitives
- Any additional objects you would like

Make sure to cover the suggested list of primitives when creating these objects, using some creativity, but limiting yourself to the techniques learned until now.

4. Illumination

Three.js technology is quite complete in terms of types of light sources, variety of materials and lighting models, which will be discussed below. In this practical work, we will be using only the `MeshPhongMaterial` class, which uses the Blinn-Phong lighting model (https://threejs.org/docs/?q=phongma#api/en/materials/MeshPhongMaterial).

4.1. Basic Light and Materials

There is one light source in the provided code, of type *PointLight*, which is characterized by emitting light equally in all directions. A *handler* (helper) is also created to provide a visual representation.

Change, in the code (identify, in the Three.js documentation, what each item represents):

- Edit the plane's **diffuse** and **specular** color to a 50% gray
- Increase the **shininess** of the plane to 100 (4.1)
- Change the color of the **point light** to a pure red (4.2)
- Revert the previous change and make the **ambient light** into pure red (4.3)

Moodle Task - PW1-C

On Moodle, discuss the following questions and submit this section's screenshots (4.1 - 4.4). Consider Phong's local illumination model as a reference, characterized by the three components: ambient, diffuse and specular.

How does the **shininess affect** the material components (ambient, diffuse, specular)?

What were the visual differences between changing the **point or the ambient light** to red?

What changed visually in the scene when the **light source was moved**? How is that change connected to the **local illumination model**?

4.2. Light Sources

So far we have been using an ambient light and an omnidirectional point light source. However, Three.js provides other types of light sources, which will be explored in this section.

All light sources extend from the **Light** class, which in turn extends from the **Object3D** class. You should check the documentation for these base classes as well to learn of all properties and functions related to the light sources.

Comment or remove the code relative to the point light, and increase the ambient light intensity to a value of 0x444444 in the constructor. You should see that the objects are minimally illuminated and without influence of any light direction.

Directional Light Source

Create an instance of a directional light source using the DirectionalLight class (documentation: https://threejs.org/docs/?q=direct#api/en/lights/DirectionalLight).

Properties for this light are:

a) Color: white,
b) Intensity; 1 cd,
c) Position: (0,10,0).

Create a helper for this new light source, and add the light source and its helper to the scene.

Manipulate the properties of this light source to see the differences:

- Change the intensity to a higher value, and then to zero
- Change the **position** of the light source to **(5, 10, 2)**.

Spot Light Source

The Spotlight source emits a cone of light, starting from the apex of the cone. For more details, refer to the Three.js documentation (https://threejs.org/docs/?q=spotl#api/en/lights/SpotLight).

Create an instance of a spotlight light (and respective *helper*) with these properties:

a) Color: white
b) Intensity: 15
c) Limit distance: 14
d) Angle of spot: 20º
e) Penumbra: 0
f) Decay: 0
g) Position: (5, 10, 2)
h) Target: (1, 0, 1)

Let's compare the directional and spotlight sources that were created:

- Set the directional light's intensity to 15, and ensure that both light sources have the same position, target, and intensity.
- Compare visually the effects of each light source by commenting out one and then the other.

Light changes in runtime

Add in the 2D interface a section to control the spotlight source, allowing you to change items a) to f) of the previous list, the Y coordinate of the position and target vector, and the visibility (`visible` property, inherited from Object3D class).

Try out the following changes:

- Change the spot angle to 35º
- Change the penumbra value to 20%, 70%, and 100%
- Disable the light's visibility
  (4.5)
- Change the position and then target's Y coordinate to 3
  (4.6)

Moodle Task - PW1-D

On Moodle, discuss the following questions and submit this section's screenshots (4.5, 4.6):

Were there any unexpected behaviors with any of the requested changes?

computer graphics as they allow adding visual realism to objects, without increasing their geometric complexity.

A mapping texture is a 2D image; its pixels are called texels but, regardless of their resolution, the textures fall into a system of U and V (or S and T…) axes, with the length U and width V of the image having a normalised value of 1.0.

In Three.js, the texture is mapped so that its bottom left corner (UV coordinates = (0,0)) is placed at the first vertex of the polygon in which the mapping is executed; also, its lower horizontal edge follows the direction of the first edge of the polygon. These rules can, however, be changed through geometric transformations of textures.

The base code made available contains some sample images in the “textures” folder to be used in these experiments. Make sure that the floor’s material is a white/gray diffuse one.

Analyse the documentation for TextureLoader, Texture, and Material for the properties and functions needed for the tasks described below:

- https://threejs.org/docs/?q=texturel#api/en/loaders/TextureLoader
- https://threejs.org/docs/?q=texture#api/en/textures/Texture
- https://threejs.org/docs/index.html?q=meshp#api/en/materials/MeshPhongMaterial

In particular, check the Texture/TextureLoader’s *load*, *wrap* and *repeat* methods and the Material’s *map* method.

In the section that creates the material of the plane object used as a floor:

- Load the floor-related texture using the TextureLoader class
- Set wrap mode to repeat in both directions (3 times in U, 4 times in V)
- Add it to the material and check the results
- Change the diffuse color of the material to pure red and observe the changes

5.1. Wrap Mode and Repeat

There are three ways to perform "wrapping". This means it is possible to control the size of the texture; in a way, this works as a "scaling" transformation to be applied to the textures.

Apply the following changes to the scene:

scene

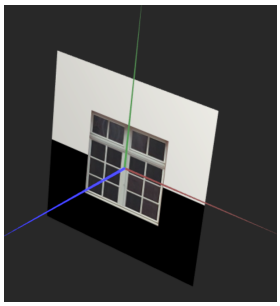
Use your creativity to take advantage of these wrap modes within your scene. You can use the textures we provide with the base code as a starting point.

5.2. Texture Offset and Rotation

As we saw, the first vertex of the polygon is by default mapped to the texture's UV = (0,0) coordinates. However, the use of *offset* values different from zero can be useful. The texture can also be rotated around the origin of the UV coordinates.

Explore this property with a new material for one of your walls:

- Create a new material and texture, using the Clamp to Edge wrap mode
    - You can use the window texture we provide in the base code
- Apply this new material to one of the walls
- Adjust the repeat and offset properties of the texture so that the window texture is centered on the wall, with a square ratio (example of the type of mapping below)
  (5.1)



- Add an element to the UI to change the wrap mode between Clamp to Edge and Repeat in run time (this may require the material to be updated: check documentation for related property).
  (5.2)
- Rotate the texture to be upside down and check the effects with Repeat mode

Moodle Task - PW1-E

On Moodle, discuss the following questions and submit this section's screenshots (5.1, 5.2):

Were you able to see changes in real time of the wrap mode? What was necessary for those

use of materials, light sources and textures.

- Diffuse materials on the walls of the room
- Specular material on table legs
- Wooden look table top
- Two paintings hanging on a wall with photographs of the group students
- A landscape in a "window" on another wall
- Add at least one of each light sources that were explored in class to the scene
- Add materials (and textures) of your choosing to the other objects

Using creativity, but limiting yourself to the techniques learned here, add other objects and effects to your liking.

Evaluation

Composition of groups: The work must be carried out in groups of three students. In case of impossibility, the best alternative must be discussed with the teacher.

Group work assessment: The code and pictures resulting from the work are to be analyzed and evaluated by the teacher. During practical classes, student involvement will be monitored. The classification given to each student may be different if there are unbalances in their contribution.

Contribute towards final grade: 10%

Criteria:

Geometry	4.0 points
Materials and textures	4.0 points
Lighting	4.0 points
Complexity and creativity	4.0 points
Answers to questionnaire in Moodle	4.0 points

Suggested workplan

- **Week of 22/Sept:**
  - Intro. to Three.js technology; Cameras; Geometric Transf.; Geometric Primitives.

class):

- Final adjustments and submission

Version 2025092222.00:00

---

[1] Helpers are built-in visual tools used for debugging and understanding a 3D scene by displaying information like axes, grids, bounding boxes, or the direction of lights and cameras